

智能家居控制系统 产品总体设计文档

产品名称：OmniHome 智能家居控制系统

文档类型：正式产品总体设计文档

版本号：V2.0

编制日期：2026 年 6 月 26 日

目录

- 1 引言
- 2 系统总体设计
- 3 系统架构设计
- 4 功能模块设计
- 5 数据设计
- 6 接口设计
- 7 运行设计
- 8 安全设计
- 9 部署设计
- 10 设计约束与演进建议
- 11 总结

1 引言

1.1 编写目的

本文档用于系统化说明 OmniHome 智能家居控制系统的总体设计方案，重点描述系统定位、设计原则、总体架构、功能模块、数据组织、接口体系、运行机制、安全控制与部署方式，为产品评审、课程答辩、后续开发与维护提供统一依据。

1.2 项目背景

随着门锁、灯光、空调、摄像头和环境传感器等智能设备不断进入家庭场景，分散式控制入口、设备状态割裂、控制反馈滞后和扩展成本偏高等问题逐步显现。OmniHome 旨在构建统一的家庭控制中枢，通过集中化交互入口和结构化业务支撑，实现设备状态感知、控制执行、场景联动与家庭协同的一体化设计。

1.3 建设目标

1. 构建统一的智能家居控制入口，降低多应用、多页面切换成本。
2. 形成从用户操作、命令执行到状态回写的完整控制闭环。
3. 支持场景与自动化能力，提升系统联动表达能力。
4. 通过共享契约、版本同步和可替换适配层保留工程化扩展空间。

1.4 适用范围

本文档面向产品设计评审、系统实现、测试验证和项目展示场景编制，描述对象为当前仓库中已实现的 OpenHarmony 客户端、Fastify 控制中心服务、SQLite 数据层和模拟设备适配层。文档不将后续可扩展能力表述为当前已落地能力。

2 系统总体设计

2.1 系统定位

OmniHome 定位为面向家庭场景的集中控制型智能家居原型系统。系统强调以统一控制中枢整合门禁、照明、环境调节、摄像头、场景与自动化等能力，并通过状态总览、命令留痕和实时推送机制支撑可演示、可验证、可演进的产品形态。

2.2 设计原则

设计原则	说明
分层解耦	将表现层、业务层、契约层、数据与设备层分离，降低耦合度。
统一契约	设备模型、命令结构、场景定义与安全信封保持单一语义来源。
状态闭环	控制链路必须覆盖执行结果、状态回写、历史记录与事件广播。
可替换扩展	模拟设备层作为可替换结构，为后续接入真实 IoT 设备保留接口。

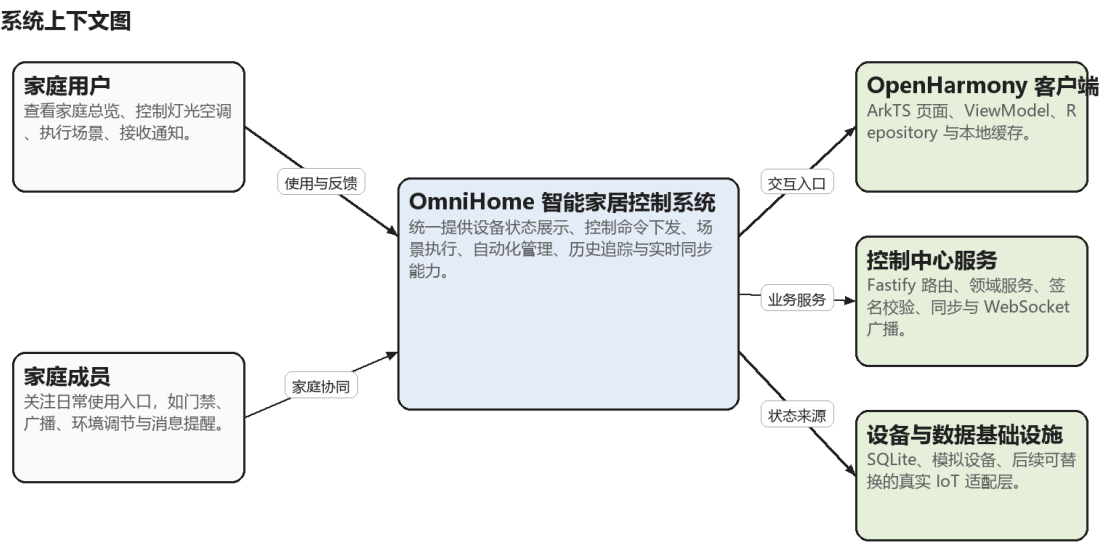


图 2-1 系统上下文图

系统总体架构图

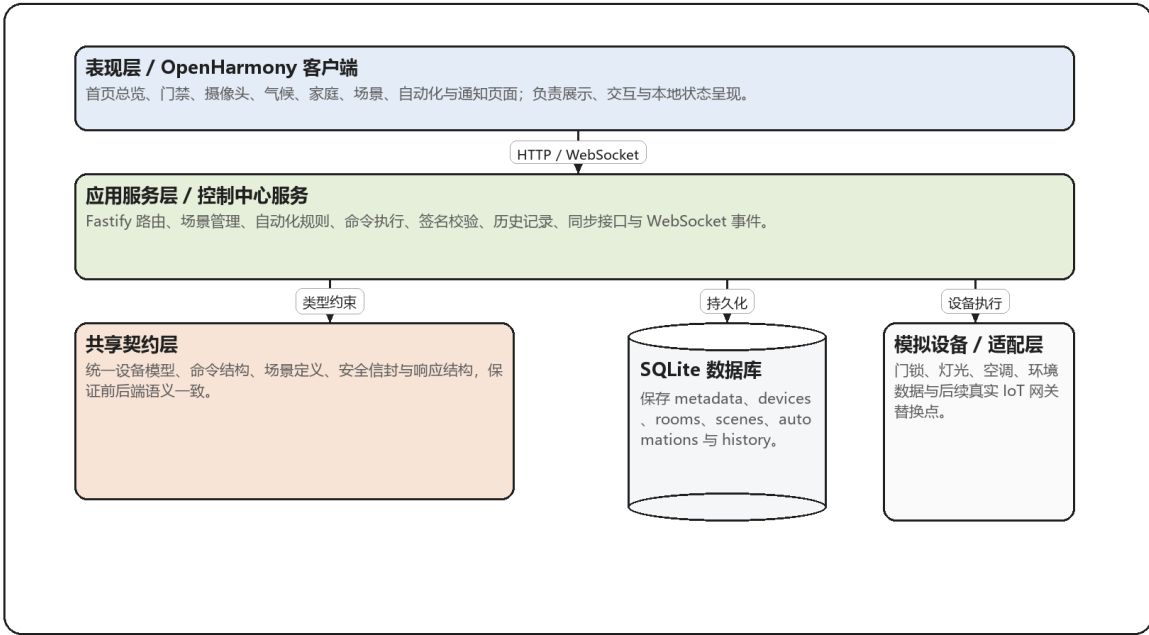


图 2-2 系统总体架构图

2.3 系统边界

当前系统边界包括 OpenHarmony 客户端、控制中心服务、共享契约包、SQLite 数据库、设备模拟层以及异常演示接口。商业级账户体系、多租户空间隔离、真实第三方物联网平台对接和云边协同治理能力属于后续演进方向，不纳入本版设计边界。

3 系统架构设计

3.1 架构概述

系统整体采用“表现层 + 应用服务层 + 共享契约层 + 数据与设备层”的四层架构。表现层负责页面展示与交互触发，应用服务层负责业务编排与规则执行，共享契约层负责统一前后端数据语义，数据与设备层负责状态来源、持久化和执行反馈。

3.2 表现层设计

OpenHarmony 客户端基于 ArkTS 页面组织业务入口，通过 ViewModel、Controller 与 Repository 抽离页面渲染逻辑、状态转换逻辑和远程数据访问逻辑，避免界面代码与网络交互和设备语义直接耦合。

3.3 应用服务层设计

控制中心服务基于 Fastify 构建，统一装配 devices、access、camera、climate、family、scenes、automations、commands、sync 和 websocket 等业务路由。该层承担命令验签、设备注册表访问、场景执行、历史记录写入、版本递增和事件广播等核心职责。

3.4 共享契约层设计

共享契约包负责统一定义设备描述、增强快照、命令结构、场景定义、命令历史和安全信封等关键模型。通过契约前置，系统可以在新增设备类型或扩展控制命令时，以“先扩展模型、再扩展实现”的方式保持演化秩序。

3.5 数据与设备层设计

数据与设备层由 SQLite、版本元数据、设备状态快照和模拟设备适配器组成。SQLite 用于支撑设备主数据、场景配置、自动化规则和命令历史的持久化，模拟设备层用于形成可验证的端到端控制闭环，并为未来替换为真实 IoT 网关保留结构位置。

4 功能模块设计

系统功能围绕“集中控制、状态感知、联动执行、家庭协同、系统支撑”五类能力展开，模块划分遵循用户认知与业务领域，而非简单按代码目录拆分。

功能模块分解图

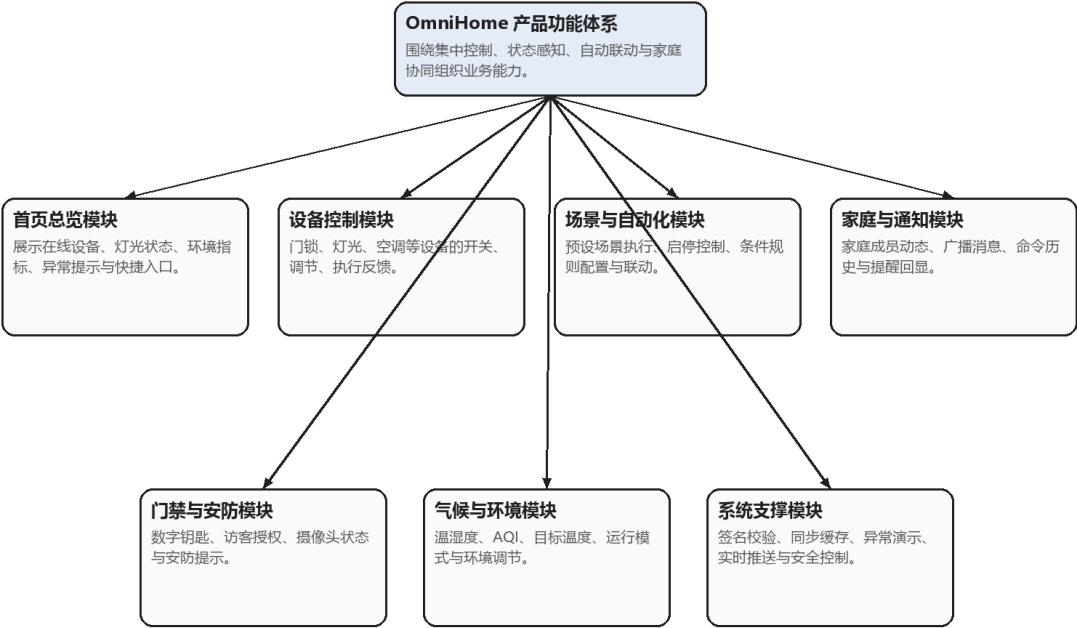


图 4-1 功能模块分解图

模块名称	主要职责	关键价值
首页总览	展示在线设备、环境数据、灯光数量与异常摘要	形成统一家庭状态入口
设备控制	门锁、灯光、空调等设备控制与状态反馈	完成高频核心控制闭环
场景与自动化	预设场景执行、规则启停与联动管理	提升家庭智能联动能力
家庭与通知	家庭成员动态、广播消息、命令历史回显	增强家庭协同与留痕
系统支撑	签名校验、同步接口、WebSocket 推送、异常演示	保障系统可靠性与可验证性

4.1 首页总览模块

首页总览模块作为系统入口页面，重点承载设备在线概览、灯光状态统计、温湿度与 AQI 摘要、风险提示和各领域页面的快捷入口。设计目标不是罗列全部细节，而是让用户在最短时间内把握家庭总体运行状态。

4.2 设备控制模块

设备控制模块面向门锁、灯光和空调等核心家居设备，统一采用“读取当前状态、发起控制请求、执行结果回显、历史记录留痕”的交互模式。该模块体现了系统最核心的业务链路，也是正式设计中必须优先保证一致性和可靠性的部分。

4.3 场景与自动化模块

场景模块负责显式触发的一组设备动作组合，自动化模块负责基于触发条件执行规则。两者在语义层区分“用户主动执行”和“系统条件触发”，既便于功能表达，也为后续扩展更复杂的规则引擎保留清晰边界。

4.4 家庭与通知模块

家庭与通知模块支撑家庭成员动态、广播消息、命令执行记录和提醒回显等能力，使系统不局限于单一设备控制，而具备家庭协同视角。该模块对于产品演示和后续产品化延展具有较高价值。

5 数据设计

5.1 数据设计目标

数据设计既服务于系统运行时状态管理，也服务于命令留痕、版本同步和可扩展性控制。当前方案采用轻量级 SQLite 作为本地持久化介质，以满足原型阶段的低部署成本和高可验证性要求。

数据库 ER 简图

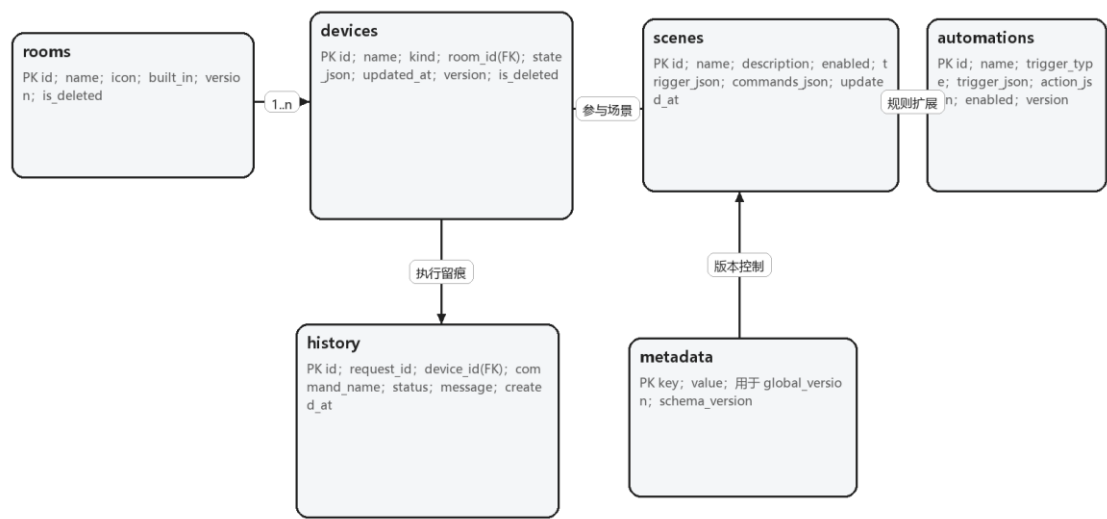


图 5-1 数据库 ER 简图

5.2 主要数据实体

数据实体	核心字段	设计说明
metadata	key、value	保存 global_version、schema_version 等元数据。
rooms	id、name、icon、version	定义房间信息，支持设备归属与页面分组。
devices	id、kind、room_id、state_json、version	记录设备快照，是总览和控制页面的数据基础。
scenes	id、name、enabled、commands_json	保存场景定义和场景动作集合。
automations	id、trigger_json、action_json、enabled	保存自动化规则及触发动作表达。
history	request_id、device_id、command_name、status	记录命令执行结果，支撑通知与审计。

5.3 版本化设计

系统通过 metadata.global_version 和各实体 version 字段实现增量同步。任一关键业务对象变更后，服务端在事务中同步提升版本号，使客户端可以只拉取发生变化的数据集合，降低全量刷新带来的资源开销和状态不一致风险。

6 接口设计

6.1 接口分层

系统接口按照业务领域进行划分，分别覆盖设备总览、命令控制、场景与自动化、垂直领域服务、同步服务和实时推送服务。这种分层方式便于客户端按模块装配能力，也便于服务端按职责演进。

接口分类	代表接口	说明
设备与摘要接口	GET /api/devices; GET /api/devices/:id; GET /api/summary	提供首页、设备详情和概览页面所需状态数据。
控制接口	POST /api/demo/sign-command; POST /api/commands; GET /api/commands/history	实现命令签名、命令执行和结果追踪。
场景与自动化接口	GET/PATCH/POST /api/scenes; GET/POST/PUT/DELETE /api/automations	支持规则查询、启停和执行。
垂直领域接口	GET /api/access; GET /api/cameras; GET /api/family; GET /api/climate	服务门禁、摄像头、家庭和气候模块。
同步与实时接口	GET /api/sync?lastVersion=n; GET /ws/events	实现按版本补偿同步与状态实时推送。

6.2 接口设计要点

1. 控制类接口采用先签名后执行的双阶段模式，避免客户端直接组装不受保护的命令报文。
2. 同步接口以版本号为中心组织增量返回结果，提升状态同步效率。
3. WebSocket 作为主动推送通道，用于补充页面轮询并缩短用户反馈时延。

7 运行设计

运行设计重点描述从用户意图触发到界面状态回显的全链路过程，覆盖控制链路、数据链路、同步链路和实时链路。

DFD 0 层数据流图

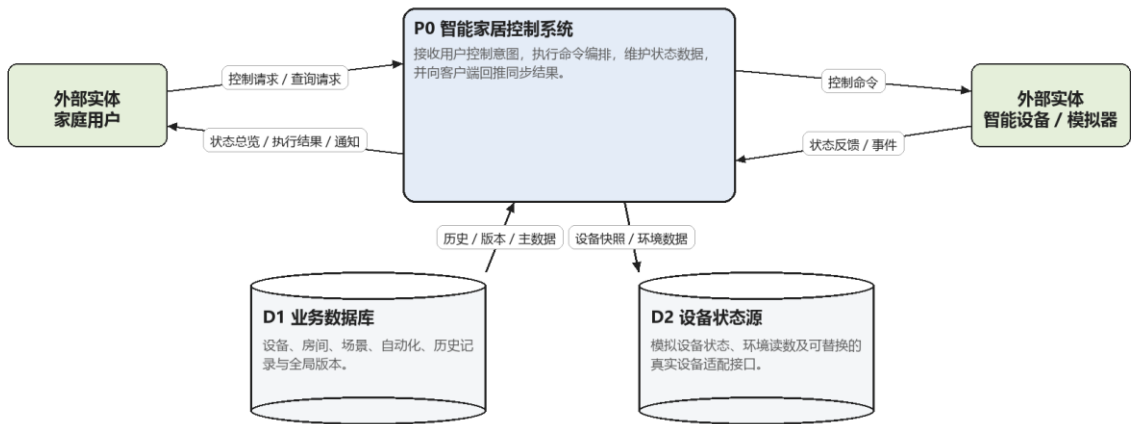


图 7-1 DFD 0 层数据流图

核心控制链路时序图

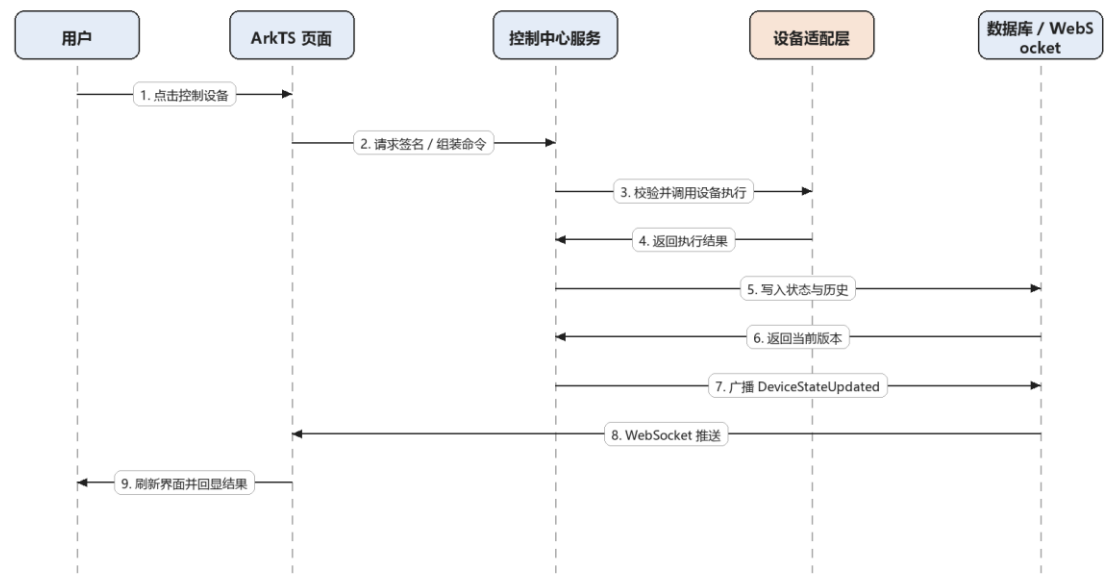


图 7-2 核心控制链路时序图

7.1 读取链路

客户端进入首页或领域页面后，通过 Repository 发起摘要查询或领域查询请求。服务端从数据库和设备状态源装配结果并返回前端，前端再将返回结构映射为页面展示模型。

7.2 控制链路

用户发起设备控制后，前端首先请求签名接口构造安全信封，再调用命令执行接口提交控制意图。服务端完成验签、重放保护、设备查找和执行后，更新数据库状态和历史记录，并将结果反馈客户端。

7.3 同步链路

客户端保存本地 lastVersion，同步接口根据该版本号返回增量变更的设备、房间、场景与自动化数据。该模式兼顾性能和一致性，是系统支撑多页面刷新与后台状态变化的关键机制。

7.4 实时链路

当设备状态变化、场景执行完成或异常演示触发时，服务端通过 WebSocket 广播事件。客户端接收事件后对本地缓存和页面模型进行局部刷新，从而实现接近实时的状态回显。

8 安全设计

8.1 命令签名机制

系统通过 HMAC-SHA256 对控制命令进行签名，安全信封包含 requestId、timestamp、nonce、payload 摘要等字段。客户端不直接伪造执行报文，而是以服务端下发的受保护结构作为命令提交载体。

8.2 重放保护机制

服务端借助 ReplayGuard 记录 nonce 和时间窗口，对重复命令和超时命令进行拒绝，降低控制链路受到重放攻击的风险。该机制对于门锁类高敏感设备控制尤为关键。

8.3 异常演示与安全反馈

系统提供安全拒绝演示接口，用于在原型环境中展示授权失败、设备离线等异常状态下的用户可见反馈。正式设计中将异常反馈纳入主链路，有助于提升系统的可解释性和可验证性。

8.4 传输安全预留

控制中心服务预留 TLS 证书与私钥配置项，支持后续切换至 HTTPS 监听模式。当前版本的重点在于将命令签名、重放保护和异常反馈等机制纳入体系化设计，而非停留在后期补丁式加固。

9 部署设计

系统采用轻量级开发与演示部署结构，由 OpenHarmony 客户端、Node.js 控制中心服务、SQLite 数据库以及模拟设备 / 适配层组成，部署成本低、链路清晰，适合课程展示与功能验证。

部署拓扑图

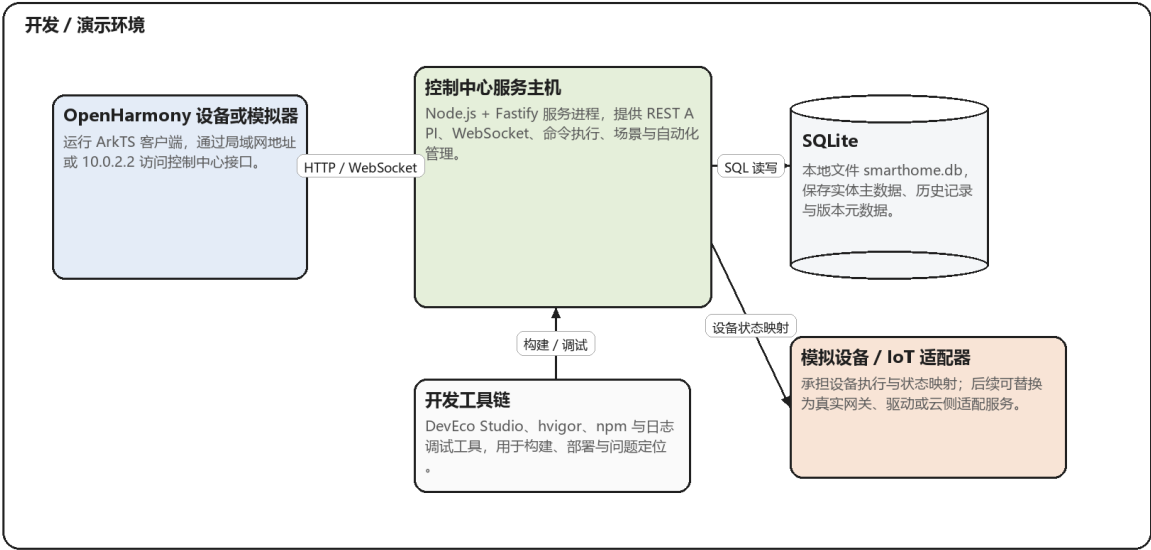


图 9-1 部署拓扑图

部署单元	运行环境	说明
OpenHarmony 客户端	模拟器或真机	运行 ArkTS 页面逻辑并访问控制中心服务。
控制中心服务	Node.js 主机进程	提供 REST API、WebSocket、签名与业务编排能力。
SQLite 数据库	服务主机本地文件	保存业务主数据、历史记录与版本元数据。
模拟设备 / 适配层	服务进程内或适配模块	用于演示设备执行与后续真实设备接入。

9.1 客户端部署说明

在模拟器环境下，客户端通常通过 10.0.2.2 访问宿主机上的控制中心服务；在局域网真机环境下，可替换为实际服务主机地址。该设计兼容演示和开发调试两类典型场景。

9.2 服务端部署说明

控制中心服务依赖 Node.js、Fastify、better-sqlite3 和共享契约包运行。服务启动后同时提供 REST API 和 WebSocket 通道，以支持页面读取、命令控制、同步补偿和实时推送等能力。

10 设计约束与演进建议

10.1 当前设计约束

1. 当前设备层主要依赖模拟器与适配抽象，尚未接入真实商业 IoT 平台。
2. SQLite 适合原型与中低并发场景，大规模并发下需要进一步演进为独立数据库服务。
3. 身份认证、权限体系和多家庭空间隔离能力尚未纳入本版正式边界。

10.2 演进建议

1. 引入真实设备网关或驱动适配层，保持上层控制接口不变，逐步替换模拟设备实现。
2. 扩展规则引擎、用户身份认证与权限控制，提升产品化落地能力。
3. 将同步、事件与审计能力进一步服务化，支撑多终端和更高复杂度场景。

11 总结

OmniHome 智能家居控制系统在当前版本中已经形成较为完整的正式产品总体设计：其一，系统边界、核心职责和分层关系明确；其二，控制闭环、同步机制和安全机制形成统一主线；其三，图示化表达覆盖了上下文、架构、功能、数据流、时序、数据实体和部署拓扑等关键视角。文档可直接作为产品设计评审、课程答辩和后续扩展实现的基础资料。